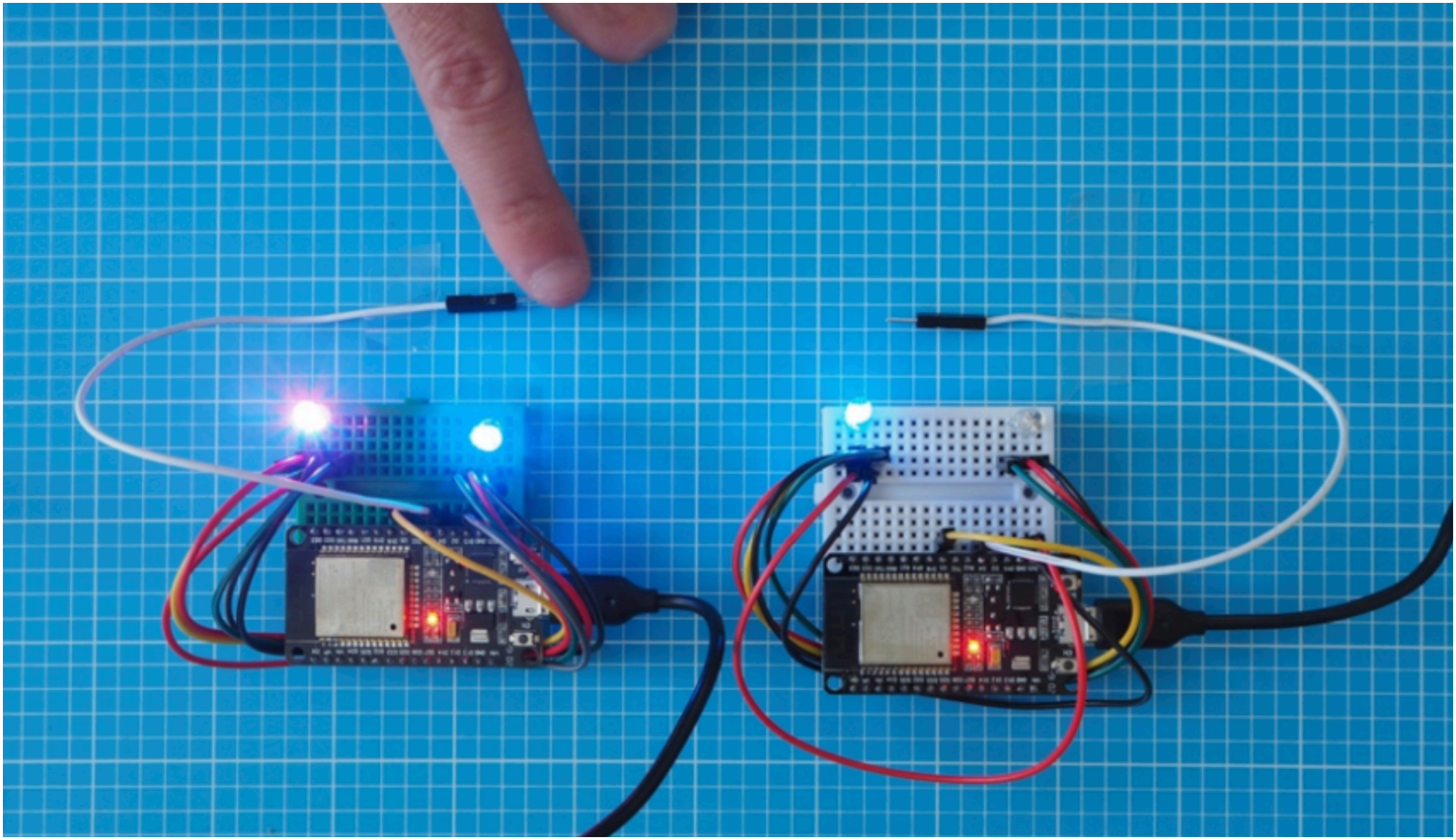
EDM2641 - Médias tangibles

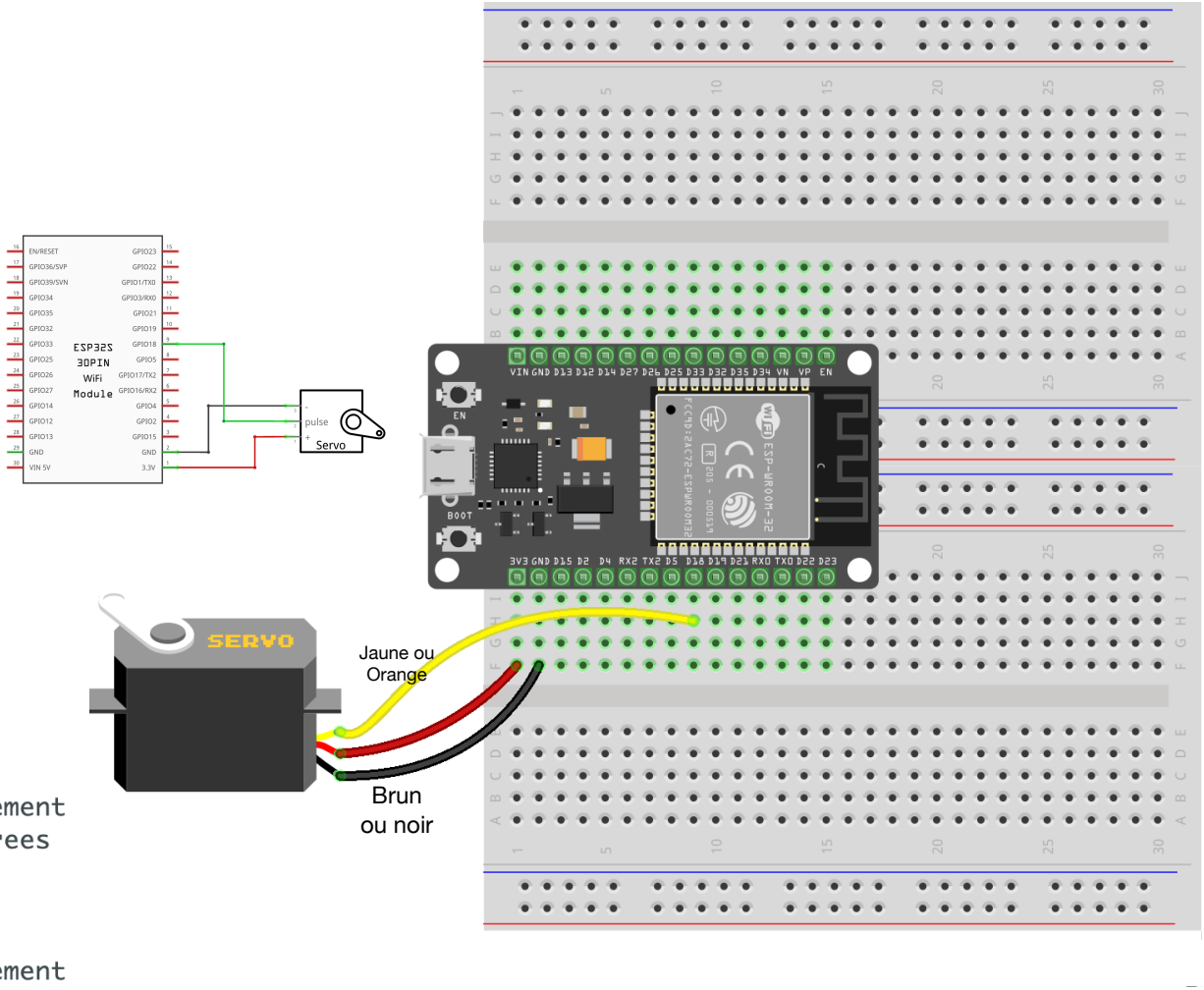
Professeur: Patrice Coulombe



Semaine 5: Servo, capteurs avancés, WiFi

Le servo-moteur






```ino
#include <ESP32Servo.h>

const int pinServo = 18;
int i;
Servo monServo;

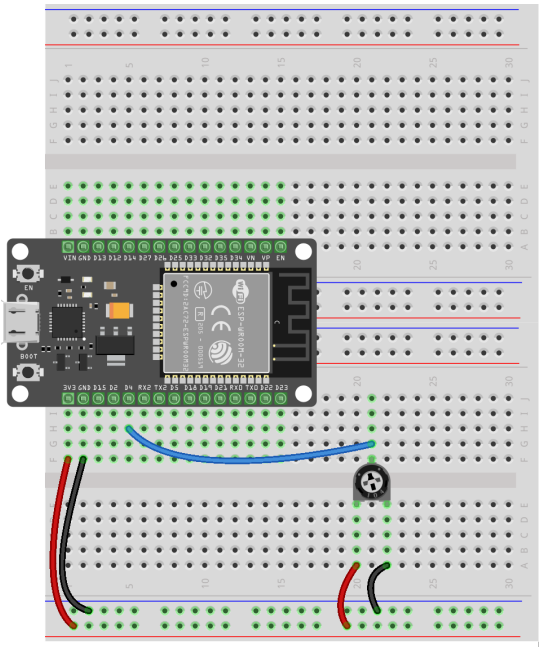
void setup()
{
  monServo.attach(pinServo);
}

void loop()
{
  for(i = 0; i<= 180; i++) //on glisse lentement
  {                        //de 0 à 180 degrees
    monServo.write(i);
    delay(20);
  }
  for(i = 0; i<= 180; i++) //on glisse lentement
  {                        //de 180 à 0 degrees
    monServo.write(180 - i);
    delay(20);
  }
}
```

Contrôle du servo via le potentiomètre (ou n'importe quel capteur!)

```
1  #include <ESP32Servo.h>
2
3  Servo monServo;
4
5  int pinServo       = 18;
6  int pinCapteur     =  4;
7  int positionServo = 0;
8  int valeurCapteur = 0;
9
10 void setup()
11 {
12   monServo.attach(pinServo);
13 }
14
15 void loop()
16 {
17   valeurCapteur = analogRead(pinCapteur);
18   positionServo = map(valeurCapteur, 0, 4095, 0, 180);
19   monServo.write(positionServo);
20 }
```

Rappel concernant le potentiomètre:



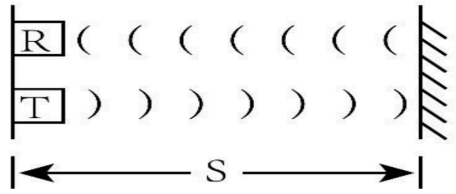
Détection de distance avec un sonar

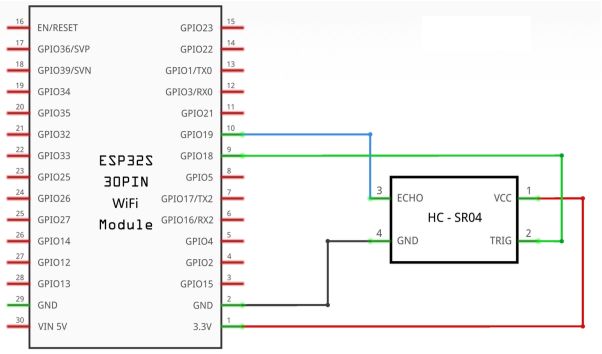
La vitesse du son est d'environ 340 m/s
L'inverse égale 0.00294 s/m (1/340)
Cela équivaut à 29.4 microsecondes par cm

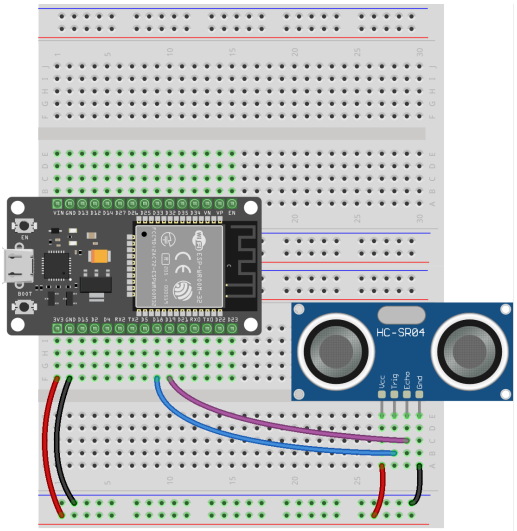
Divisez la sortie de pulsin(), qui est par microsecondes, par 29.4 pour convertir le temps en distance.

Puisque le résultat correspond à un aller-retour du son (un écho), diviser ce résultat par 2.

Pour faire plus simple, divisez la sortie de pulsin() par 58.8 car 58.8 = 29.4 x 2








```
1  const int TrigPin = 18;
2  const int EchoPin = 19;
3  float distance;
4
5  void setup()
6  {
7    Serial.begin( 115200);
8    pinMode(TrigPin, OUTPUT);
9    pinMode(EchoPin, INPUT);
10
11   digitalWrite(TrigPin, LOW);
12 }
13
14 void loop()
15 {
16   digitalWrite(TrigPin, HIGH);
17   delayMicroseconds(10);
18   digitalWrite(TrigPin, LOW);
19
20   distance = pulseIn(EchoPin, HIGH) / 58.8;  //echo converti en cm
21
22   Serial.println(distance); //en centimetres
23   delay(50);
24 }
```

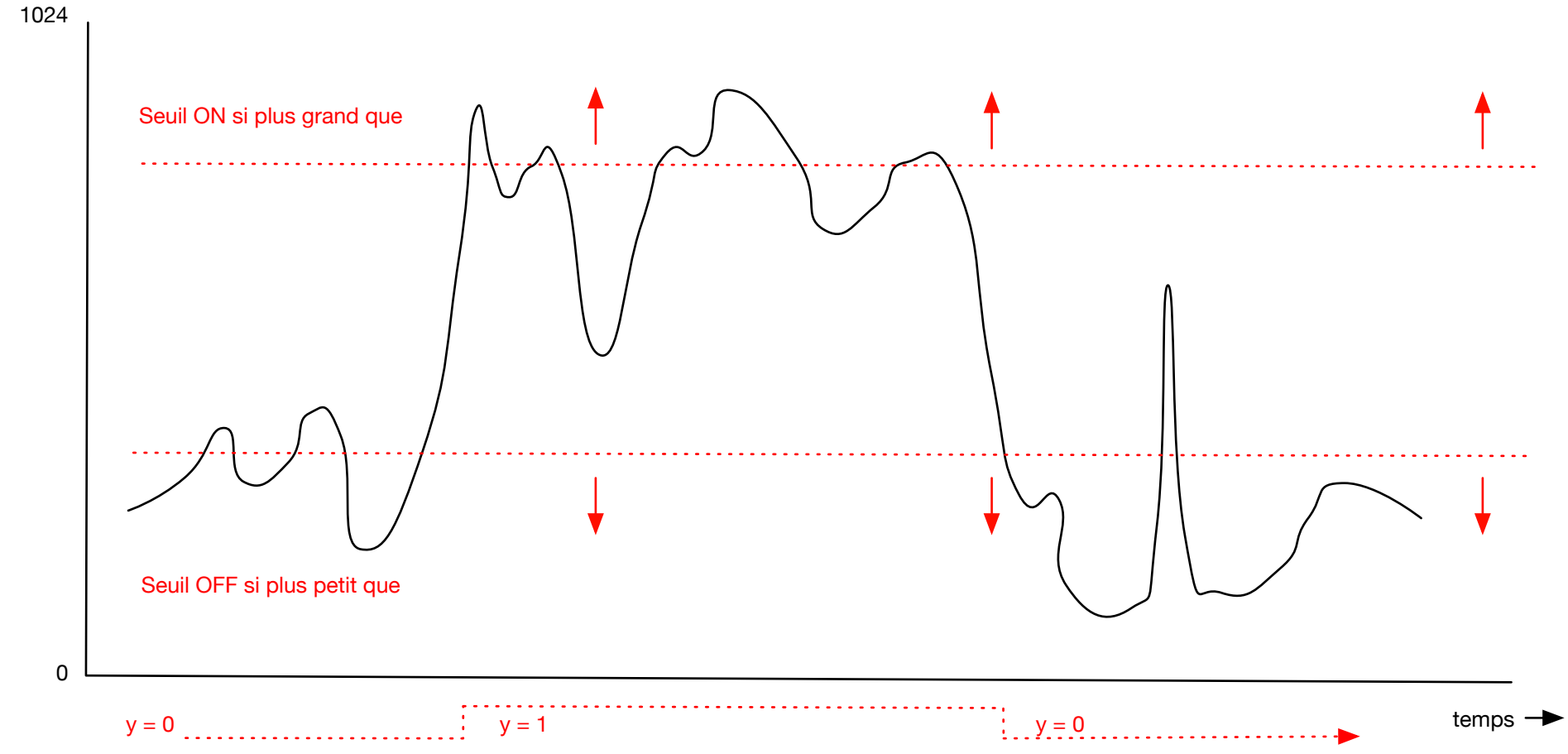
Seuil Hysteresis

La méthode idéale pour produire un seuil de détection avec le signal d'un capteur

Sortie du Seuil (y) :

Si x > ON, alors y = 1
Si x < OFF, alors y = 0
Sinon, alors y = y

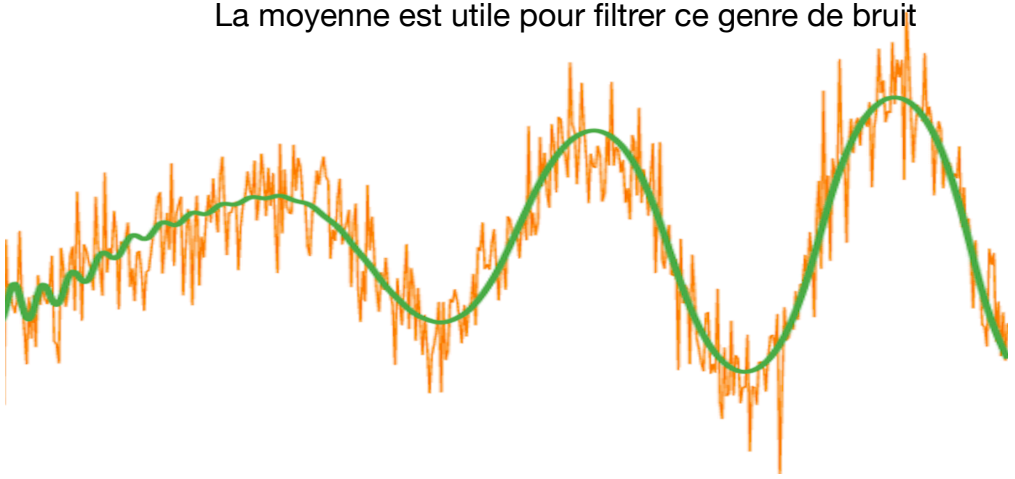
```
1  bool hysteresis(int x, bool precedent_etat, int seuil_ON, int seuil_OFF)
2  {
3      if(x <= seuil_OFF)
4        return 0;
5      else if(x >= seuil_ON)
6        return 1;
7      else
8        return precedent_etat;
9  }
10
11 //usage de la fonction
12 bool etat = 0;
13
14 void loop()
15 {
16   etat = hysteresis(analogRead(A0), etat, 500, 400);
17 }
```



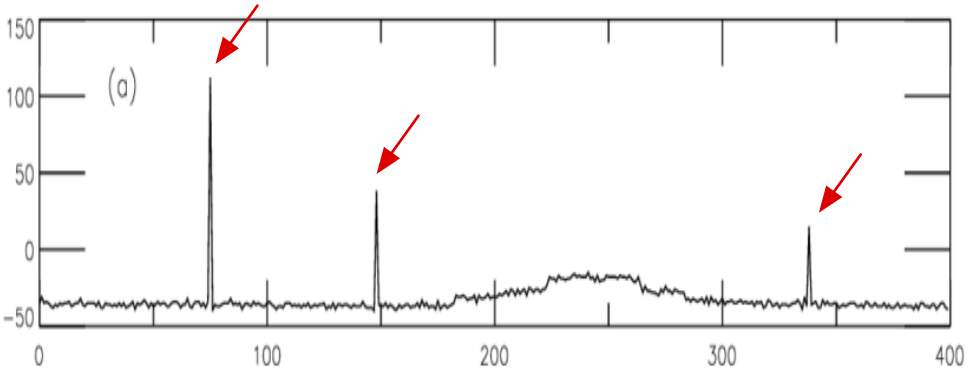
Lissage d'un signal (moyenne et médiane)

```
1  #include <RunningMedian.h>
2
3  RunningMedian signalCapteur = RunningMedian(15);
4
5  void setup()
6  {
7    Serial.begin(57600);
8  }
9
10 void loop()
11 {
12   int x = analogRead(A0);
13
14   signalCapteur.add(x);
15   long m = signalCapteur.getMedian();
16   long a = signalCapteur.getAverage();
17
18   Serial.print(x);
19   Serial.print(" ");
20   Serial.print(a);
21   Serial.print(" ");
22   Serial.println(m);
23   delay(100);
24 }
```

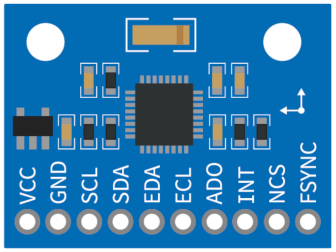
La moyenne est utile pour filtrer ce genre de bruit



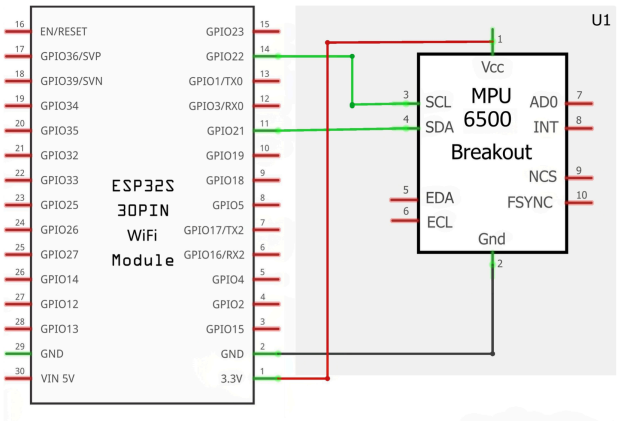
La médiane est utile pour filtrer ce genre de glitch



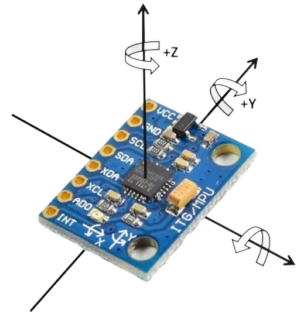
Accéléromètre / Gyroscope et communication i2C

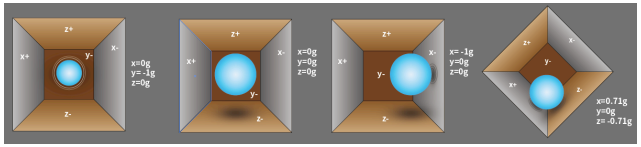


MPU6500
(cousin du MPU6050)

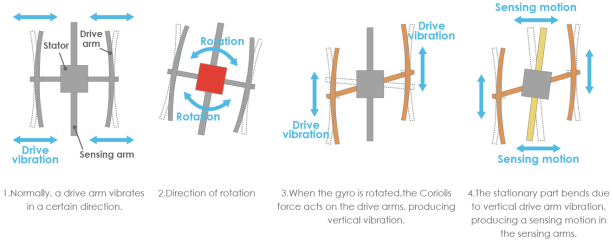


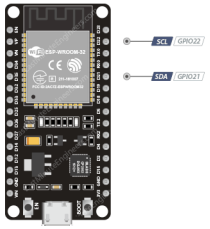
Accéléromètre

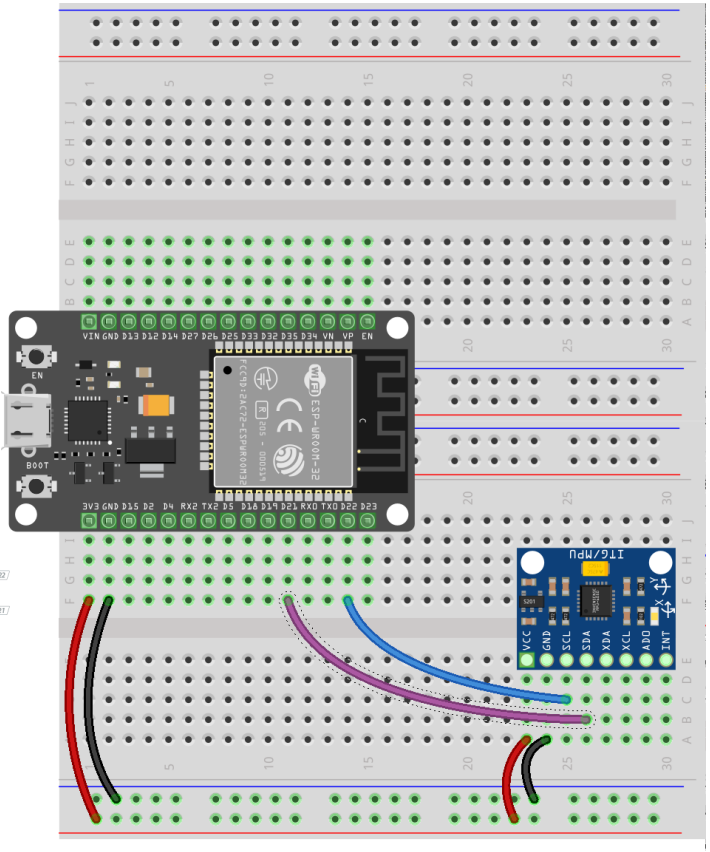




Gyroscope







Programme pour l'accéléromètre/gyroscope (mpu6500)

Pour vous simplifier la vie, vous pouvez utiliser des librairies dédiées au MPU6500 !
Il y a plusieures façons d'interpréter et calibrer ce capteur.

```ino
#include<Wire.h>
const int MPU_addr=0x68;  // I2C address of the MPU-6050

int16_t AcX,AcY,AcZ,Tmp,GyX,GyY,GyZ;

void setup()
{
  Wire.begin(21,22);
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x6B);  // PWR_MGMT_1 register
  Wire.write(0);     // set to zero (wakes up the MPU-6050)
  Wire.endTransmission(true);
  Serial.begin(115200);
}

void loop()
{
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);  // starting with register 0x3B (ACCEL_XOUT_H)
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_addr,14,true);  // request a total of 14 registers

  AcX=Wire.read()<<8|Wire.read();  // 0x3B (ACCEL_XOUT_H) & 0x3C (ACCEL_XOUT_L)
  AcY=Wire.read()<<8|Wire.read();  // 0x3D (ACCEL_YOUT_H) & 0x3E (ACCEL_YOUT_L)
  AcZ=Wire.read()<<8|Wire.read();  // 0x3F (ACCEL_ZOUT_H) & 0x40 (ACCEL_ZOUT_L)
  Tmp=Wire.read()<<8|Wire.read();  // 0x41 (TEMP_OUT_H) & 0x42 (TEMP_OUT_L)
  GyX=Wire.read()<<8|Wire.read();  // 0x43 (GYRO_XOUT_H) & 0x44 (GYRO_XOUT_L)
  GyY=Wire.read()<<8|Wire.read();  // 0x45 (GYRO_YOUT_H) & 0x46 (GYRO_YOUT_L)
  GyZ=Wire.read()<<8|Wire.read();  // 0x47 (GYRO_ZOUT_H) & 0x48 (GYRO_ZOUT_L)

  Serial.print("AcX = "); Serial.print(AcX);
  Serial.print(" | AcY = "); Serial.print(AcY);
  Serial.print(" | AcZ = "); Serial.print(AcZ);
  Serial.print(" | Tmp = "); Serial.print(Tmp/340.00+36.53); //temperature formula
  Serial.print(" | GyX = "); Serial.print(GyX);
  Serial.print(" | GyY = "); Serial.print(GyY);
  Serial.print(" | GyZ = "); Serial.println(GyZ);

  delay(50);
}
```

Communication OSC via un réseau WiFi

ESP32 en mode **Hotspot**.
C'est-à-dire qu'il devient une borne WiFi.
Il transmet un message **OSC** contenant la valeur d'un capteur de touché.

Le WiFi accapare les ressources de **ADC2**: C'est-à-dire que la fonction *analogRead()* n'est plus disponible pour les **GPIO 2, 4, 12, 13, 14, 15, 25, 26 et 27**.
Mais, étonnamment, toutes les pins **TOUCH** pour *touchRead()* fonctionnent bien!

Pour *analogRead()* , vous pouvez utiliser les **GPIO 32 à 39** sans souci (**ADC1**)

```ino
#include <WiFi.h>
#include <WiFiUdp.h>    // Réseau UDP
#include <OSCMessage.h> // Protocole OSC

char hotspotName[] = "patou_hotspot";
char password[] = "12345678"; //minimum 8 caractères

WiFiUDP Udp;
const IPAddress IP_ordinateur(192, 168, 4, 2);//Addresse de l'ordinateur
const int outPort = 9999;  // Port que l'ordinateur écoute

void setup() {
  WiFi.softAP(hotspotName, password); // Création du Hotspot
  Udp.begin(outPort);      // Start UDP
}

void loop() {
  int valCapteur = touchRead(15);

  OSCMessage msg("/capteur1");
  msg.add(valCapteur); //Message: " /capteur1 <int> "

  Udp.beginPacket(IP_ordinateur, outPort);
  msg.send(Udp);
  Udp.endPacket();
  msg.empty(); // On vide l'espace mémoire
  delay(50); //Cadence du message (période d'échantillionnage)
}
```